

# Project Report: AstroRL

Daniil Khoma, HAW Hamburg

**Abstract—** This project investigates the application of reinforcement learning to a simplified orbital transfer problem. A custom physics-based simulation environment was developed to model spacecraft motion under Newtonian gravitational dynamics with discrete thrust actions. Two policy-gradient methods were implemented using PyTorch: the Monte Carlo REINFORCE algorithm and the Advantage Actor–Critic (A2C) method.

For each algorithm, ten independent training runs were conducted. Performance was evaluated using episode reward statistics, success rates, and convergence metrics. While both methods demonstrated measurable learning behavior, neither consistently achieved stable positive reward regimes or reliable orbital transfer trajectories. The Actor–Critic method showed faster convergence and improved stability compared to REINFORCE, but still exhibited high variance across runs.

The results suggest that baseline policy-gradient methods can partially learn structured control behaviors in orbital settings, yet remain sensitive to reward design, state representation, action parameterization, and hyperparameter selection. The developed framework provides a modular foundation for future extensions, including improved environment modeling and more advanced policy optimization algorithms such as Proximal Policy Optimization (PPO).

**Index Terms—** Reinforcement Learning, Monte Carlo REINFORCE, Advantage Actor–Critic, N-Body Simulation, Astrodynamics Simulation, Orbital Transfer, Spacecraft Guidance and Control.

## I. INTRODUCTION

As space technology continues to advance, space missions are becoming more frequent and increasingly complex. These missions rely on precise orbital maneuvers to reach target orbits, perform rendezvous operations, or adjust mission trajectories. Traditionally, such maneuvers are planned using analytical methods and optimal control techniques that assume accurate system models and well-defined mission constraints.

With growing interest in autonomous spacecraft and long-duration missions, there is increasing motivation to investigate data-driven approaches that can operate under uncertainty and adapt to changing conditions. Reinforcement learning offers a framework for learning control policies through interaction with a simulated environment and has shown promising results in other control and robotics applications.

This project investigates the applicability of reinforcement learning methods to orbital maneuver planning within a physics-based simulation. The focus lies on evaluating whether learned policies can achieve feasible and efficient orbital transfers and how their behavior compares under different learning algorithms and reward formulations.

### Context

Currently, the main challenge for the exploration of distant planets in the solar system is the very long flight time combined with the absence of infrastructure such as refueling or maintenance stations along the route. This implies that all propellant and all necessary

This project was carried out as part of the Self-Optimizing Systems module at HAW Hamburg, Faculty Computer Science and Digital Society, winter semester 2025/26.

equipment must be carried from launch, either from Earth’s surface or from Earth orbit. The extremely high cost of lifting mass out of Earth’s gravity well further amplifies this constraint, so every gram of propellant and every cubic centimeter of spacecraft volume must be justified. As a result, trajectory design is typically driven by two primary objectives: minimizing mission duration and maximizing propellant efficiency (or, equivalently, minimizing changes in velocity vector  $\Delta v$ ). To achieve this, numerous methods for calculating high precision trajectories under limitations exist:

- Hohmann transfers and variants: A Hohmann transfer is the analytically optimal two-impulse transfer between coplanar circular orbits. It uses an elliptical transfer orbit tangent to both the initial and target orbits [1]. In interplanetary mission design, Hohmann-like reasoning provides first-order estimates of required velocity and time of flight (e.g. simple Earth–Mars transfers) before higher-fidelity optimization.
- Lambert solvers / Lambert guidance: Lambert’s problem is the boundary-value problem of finding a Keplerian orbit that connects two position vectors in a given time of flight [2]. Its solution provides the required initial and final velocity vectors. Modern trajectory design tools and libraries use fast Lambert algorithms as the inner engine for generating candidate legs in patched-conic and global-optimization schemes for interplanetary missions.
- Patched-conic methods and global optimization. In patched-conic methods, the spacecraft trajectory is modeled as a sequence of Keplerian arcs in different gravitational spheres of influence, which are “patched” together at the boundaries. Iterative variants for multi-flyby interplanetary trajectories repeatedly solve Lambert problems and adjust flyby conditions to obtain a smooth, dynamically consistent route [3]. Global optimization techniques such as genetic algorithms or Monte Carlo Tree Search typically vary departure dates, flyby sequences, and times of flight, while calling a Lambert solver to construct each leg and evaluate the resulting  $\Delta v$ , constraints, and mission objectives. The Cassini mission to Saturn and multistage Artemis II mission are validation cases for such methods [4], [5].

More recently, machine-learning methods have been investigated as an alternative or complement to classical optimal-control techniques. In particular, reinforcement learning has been applied to the design of robust low-thrust interplanetary transfers and generic orbital maneuvers by formulating the transfer as a Markov decision process and training neural-network policies with algorithms such as Proximal Policy Optimization [6]. Transformer-based reinforcement-learning architectures for multi-phase trajectory control have also been proposed, aiming to handle complex mission profiles [7].

### Project question

Reinforcement learning (RL) -based navigation and control methods are currently an active research topic, and first applications are already being tested in real space-flight scenarios. However, they do not replace classical trajectory-optimization techniques, but extend them with data-driven decision-making and closed-loop guidance that can adapt online to changing conditions. This makes RL particularly

interesting for missions where the environment is uncertain, the dynamics are complex, or the control has to react autonomously to unforeseen events over long timescales.

Compared to traditional analytical and numerical methods, RL offers several potential advantages.

- 1) Once a policy has been trained, generating control actions is extremely fast, which makes it attractive for on-board guidance in real time.
- 2) RL can be trained directly in high-fidelity simulators that include perturbations, sensor noise, partial observability, or actuator limitations, so that the learned policy implicitly handles effects that are difficult to capture in closed-form models.
- 3) RL methods can naturally optimize complex, multi-objective reward functions (for example combining fuel efficiency, mission time, safety margins, and hardware constraints) without the need to hand-craft a new optimal-control formulation for every mission scenario.

As part of the lecture “Self-Optimizing Systems” at HAW Hamburg, a number of modern machine learning methods, with a focus on reinforcement learning, were introduced. This led to the question of whether RL can be applied not only to optimize isolated subproblems, but to learn complete maneuver trajectories under realistic orbital dynamics. In particular, whether such approaches remain viable when realistic physical dynamics and operational constraints are taken into account.

Based on this motivation, the central research question of this work is whether reinforcement learning methods are capable of generating feasible and efficient orbital maneuver strategies within a physics-based simulation, and how their performance compares across different algorithmic choices and reward formulations. This work does not aim to replace classical trajectory optimization methods, but to explore the feasibility and behavior of reinforcement learning approaches in a simplified, controlled simulation setting.

## II. PROBLEM FORMULATION

### Objective

To investigate the applicability of reinforcement learning to orbital maneuver planning, a set of simulation environments with increasing complexity was initially conceptualized. The intention was to incrementally extend the task difficulty while maintaining a consistent problem structure, thereby enabling a structured investigation into how reinforcement learning algorithms may handle progressively more demanding dynamical systems.

This staged approach was designed to first evaluate whether policy-based methods can reproduce physically meaningful orbital transfers in simplified settings and, if successful, to examine their behavior under more complex and realistic conditions.

As a baseline, a simplified two-body environment was defined in which the agent performs a transfer between two circular, coplanar orbits. Since the optimal solution for this maneuver is analytically known (i.e., the classical Hohmann transfer), this scenario provides a physically interpretable reference case for algorithmic evaluation and validation.

More advanced environments were conceptually outlined, including extended transfer configurations and preliminary multi-body setups intended to approximate gravity-assist maneuvers. These scenarios introduce additional challenges such as longer time horizons, higher sensitivity to initial conditions, and increasingly sparse reward signals.

However, due to the computational demands, time constraints and stability challenges inherent to reinforcement learning in continuous orbital dynamics, the experimental evaluation presented in this work

focuses on the two-body Hohmann transfer scenario. Concentrating on this controlled baseline enables a systematic analysis of algorithm behavior, training stability, and reward design within the scope of the project. Environmental uncertainties and perturbations are not explicitly modeled. The broader objective remains to explore the limitations and characteristics of reinforcement learning methods in orbital control contexts, rather than to develop operational mission-planning solutions.

### Observation Space

In the initial experimental setups, the observation space provided to the agent is defined relative to the target orbit or target object. Specifically, the agent observes the relative position and velocity vectors with respect to the target, as well as the corresponding Euclidean distance, relative speed magnitudes and orientation. For simple orbital transfer tasks, such as a Hohmann transfer between circular and coplanar orbits, this relative-state formulation is hypothesized to be sufficient for baseline transfer scenarios. Using relative quantities also reduces sensitivity to absolute coordinate choices and improves generalization across different initial conditions, while keeping the dimensionality of the observation space low.

However, it is not clear whether this observation space remains sufficient for more complex mission scenarios, such as transfers involving multiple bodies, flybys, or long-term trajectory shaping. In such cases, additional information, for example absolute orbital parameters, time-dependent features, or properties of secondary bodies, may be required to fully capture the relevant system dynamics. The suitability of the chosen observation space is therefore evaluated implicitly through the agent’s learning performance across environments of increasing complexity.

### Action Space

The action space is defined as a five-dimensional control vector. Four action components correspond to thrust commands along fixed directions in the global reference frame, while the fifth action represents a null action in which no thrust is applied. The exact mapping between action values and resulting accelerations depends on the specific implementation of the controlled spacecraft model and simulation parameters, including thrust limits, mass properties and time discretizations. This action formulation provides a simple and consistent interface across different environments, while allowing the learning agent to generate continuous or discrete thrust commands depending on the underlying control model. For the initial environments, this abstraction is hypothesized to be sufficient for basic orbital transfers.

As with the observation space, the adequacy of this action representation for more complex mission scenarios is not guaranteed. More advanced tasks may require alternative action parameterizations. These include thrust direction expressed in a local orbital frame, variable burn durations, or variable thrust. These extensions are considered beyond the scope of the initial experiments but represent natural directions for future refinement of the environment design.

### Termination Conditions

Each episode begins with a predefined initial orbital state and is terminated when one of several conditions is met. An episode is considered successful if the spacecraft reaches a state sufficiently close to the target orbit, as defined by thresholds on relative position and velocity. Episodes are terminated unsuccessfully if a maximum simulation time is exceeded or if operational constraints are violated. The constraints include, but are not limited to, system escape,

collision with another simulation object, or numerical instability of the simulation. These termination criteria ensure a finite learning horizon while encouraging the agent to reach the target efficiently.

### Reward Function

The reward function is designed to guide the learning process toward feasible and efficient orbital maneuvers. A positive terminal reward is assigned when the target orbit is reached, while penalties are applied for fuel consumption and elapsed time in order to discourage unnecessarily long or inefficient trajectories. Additional shaping terms are employed in the initial environments, contingent on the relative distance and velocity with respect to the target. These terms are intended to stabilize learning and provide intermediate feedback. The reward structure is implemented across environments, where feasible, with modifications introduced when necessary to accommodate increased task complexity.

### Evaluation metrics

The primary evaluation focus in this project is on learning performance, since the current experiments aim to assess whether reinforcement learning algorithms can acquire meaningful control behavior in realistic scenario. Physically meaningful orbital performance metrics, such as deviation from analytically computed trajectories, are reserved for future work once policies demonstrate stable learning. The following metrics are used to evaluate the reinforcement learning agents:

- **Average episode return:** Mean cumulative reward per episode, reflecting how well the agent optimizes the shaped reward function over time.
- **Success rate:** Fraction of episodes in which the agent achieves the terminal orbital tolerances, signaling successful orbit insertion.
- **Convergence speed:** Number of training episodes required to reach a stable policy or a target success rate, indicating sample efficiency.
- **Training stability:** In addition, variability across independent training runs is analyzed to assess robustness and training stability.

If an agent demonstrates consistent learning and achieves meaningful control behavior, subsequent evaluation against classical trajectory solutions, such as the analytical Hohmann transfer, can be performed to quantify orbital accuracy, fuel efficiency, and maneuver time.

## III. SIMULATION ENVIRONMENT

Trajectory optimization problems in astrodynamics are inherently tied to the accurate modeling of gravitational  $N$ -body dynamics. While simplified two-body Keplerian models suffice for basic orbital maneuvers, complex mission scenarios, such as long-duration interplanetary transfers or multi-flyby gravity-assist trajectories, require the consideration of multi-body gravitational effects. To enable the investigation of increasingly realistic orbital dynamics, this work employs an  $N$ -body simulation environment as the foundation for the reinforcement learning setup.

The custom gravitational  $N$ -body simulator implemented for this project was heavily inspired by the practical tutorial materials of Christoph Schäfer [8]. These resources provided the theoretical foundation for the underlying physics and the numerical integration methods

### Orbital Dynamics Model

In the simulator the motion of each body is governed by Newton's law of universal gravitation given by

$$F = G \frac{m_1 m_2}{r^2}, \quad (1)$$

where  $F$  is the magnitude of the gravitational force between two point masses  $m_1$  and  $m_2$ ,  $r$  is the distance between their centers of mass, and  $G$  is the gravitational constant.

The acceleration of each body is computed from the pairwise gravitational interactions with all other bodies in the system. This approach allows the simulation to represent both simplified two-body scenarios and more complex multi-body configurations within a unified framework.

In this project, the dynamics are restricted to two-dimensional motion. This reduction simplifies visualization and analysis while preserving the essential characteristics of orbital mechanics relevant to the investigated maneuver tasks. All bodies are modeled as point masses, and non-gravitational perturbations such as atmospheric drag or radiation pressure are neglected.

### Numerical Integration Method

Since closed-form solutions are generally not available for  $N$ -body systems [9], the equations of motion are solved numerically. Two integration schemes were implemented and evaluated: the symplectic Euler method and the velocity Verlet integrator.

The symplectic Euler method updates velocity first and then position:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t \mathbf{a}(\mathbf{x}_n), \quad (2)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t \mathbf{v}_{n+1}. \quad (3)$$

The velocity Verlet integrator is defined as

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t \mathbf{v}_n + \frac{1}{2} \Delta t^2 \mathbf{a}(\mathbf{x}_n), \quad (4)$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \frac{\Delta t}{2} [\mathbf{a}(\mathbf{x}_n) + \mathbf{a}(\mathbf{x}_{n+1})]. \quad (5)$$

Both methods are commonly used in physics-based simulations due to their favorable stability properties when applied to Hamiltonian systems [8]. The symplectic Euler scheme provides a computationally efficient first-order method with bounded long-term energy error, making it suitable for rapid experimentation and large numbers of simulation steps. The velocity Verlet method, while slightly more computationally expensive, achieves second-order accuracy and typically exhibits reduced energy drift over extended simulation horizons.

Comparing these integrators enables an assessment of the trade-off between computational efficiency and numerical fidelity in the context of reinforcement learning training.

### Numerical Validation and Stability Analysis

To assess the correctness and numerical stability of the simulation, several diagnostic quantities were monitored during extended simulation runs. These include the conservation of total linear momentum and total angular momentum of the system, as well as the drift of the system's center of mass. In addition, the total mechanical energy of the system was tracked to evaluate long-term energy conservation properties of the numerical integrators.

These validation tests provide insight into the numerical behavior of the simulation and help identify integration-induced artifacts such as secular energy drift. Monitoring these quantities is particularly important in reinforcement learning experiments, where small numerical errors can accumulate over long time horizons and potentially

influence the learning process. The results of these tests were used to guide the selection of integration parameters and to ensure that observed agent behavior arises from the learning setup rather than from simulation instability.

#### Qualitative Behavior and Visualization

To evaluate the quality of the simulator and the implemented integrators, a configuration of three identical objects in a stable orbit (one of the stable solutions to the three-body problem [10]) was used. Visualization of the trajectories and analysis of the stability of the system showed that the simulator calculates physically plausible orbital motions without collapse or artificial divergence over a long simulation time, given the initial configuration leads to a stable orbit. The data analysis and their visualizations can be found in the appendix I.

### IV. REINFORCEMENT LEARNING METHODS

For this project, multiple reinforcement learning algorithms were considered to solve control tasks. Initially, a Monte Carlo policy gradient method based on the REINFORCE algorithm was implemented. The policy is represented by a neural network that maps the current state of the system to a distribution over discrete actions. Complete trajectories are generated by interacting with the environment, and after each episode, the cumulative return at each time step is used to update the policy parameters. A second algorithm is based on the Actor–Critic framework. In this method, the neural network not only represents the policy but also provides an estimate of the expected return for each state. This allows the policy updates to be guided, reducing variance and enabling more frequent updates during an episode, which may be beneficial for orbital control tasks. Due to time constraints, more modern, complex, and potentially better-suited algorithms could not be implemented, so the focus remained on these two approaches as representative methods for the problem.

#### Monte Carlo Policy Gradient

The Monte Carlo policy gradient method, also referred to as the Monte Carlo REINFORCE method, was chosen because it provides a simple baseline for environments with continuous state spaces and discrete action spaces. Being policy-based and model-free, it allows direct optimization of the policy without additional components such as target networks or replay buffers, making it straightforward to implement and analyze in the initial stage of the project. By relying on full trajectory returns rather than bootstrapped estimates, the algorithm also ensures unbiased gradient updates.

However, this approach has important limitations in the context of the project. Updates are performed only after complete episodes, which can make learning slow for long trajectories typical in orbital simulations. The variance of gradient estimates is high, leading to unstable training and sensitivity to reward scaling or small action effects. These factors can hinder convergence to optimal orbits and complicate fine control, motivating the exploration of variance-reducing and bootstrapped methods in subsequent algorithm choices.

**Policy Network:** The Monte Carlo agent is implemented using the PyTorch library [11]. The policy is represented by a feedforward neural network

$$\pi_{\theta}(a | s) \quad (6)$$

which maps the current state observation  $s$  to a probability distribution over discrete thrust actions  $a$ . The policy network parameters  $\theta$  consist of:

- An input layer with dimensionality equal to that of the observation vector (see Section V)

- Two fully connected hidden layers with 128 units each
- ReLU activation functions
- An output layer matching the number of discrete actions (see Section II)

The output layer produces unnormalized logits that define a categorical distribution over actions. During training, actions are sampled from this distribution, ensuring stochastic exploration.

**Policy Update:** After each episode, the discounted return  $G_t$  is computed for every time step.

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k} \quad (7)$$

where  $\gamma$  is the discount factor and  $r_i$  is the reward for the  $i$ -th step.

The policy parameters are then updated by increasing the probability of actions that yielded high returns and decreasing the probability of actions that resulted in low returns. This is implemented by minimizing the loss  $\mathcal{L}$ :

$$\mathcal{L} = - \sum_t \log \pi_{\theta}(a_t | s_t) \hat{G}_t \quad (8)$$

where  $\hat{G}_t$  denotes the normalized return. The loss minimization is implemented via Adam optimization.

#### Advantage Actor–Critic

To address the limitations of the Monte Carlo policy gradient method, an Actor–Critic approach was considered as the next step. In this framework, the policy (actor) is still represented by a neural network mapping states to action distributions, but an additional neural network (critic) is introduced to estimate the expected return from each state. During training, the critic provides a baseline that reduces the variance of the policy gradient, allowing the actor to receive more stable and informative updates.

The Advantage Actor–Critic method is seen as suitable for the orbital control task because it combines the advantages of policy-based learning with bootstrapped value estimates. By partially relying on the critic’s predictions, learning can occur at each time step rather than only after complete episodes, in theory significantly accelerating convergence in long-horizon simulations. This approach maintains compatibility with continuous state spaces and discrete actions while improving stability and control precision compared to the purely Monte Carlo method.

**Policy and Value Networks:** The actor is represented by the same feedforward policy network as in Monte Carlo method (see Section IV) In addition, a separate value network (critic) is introduced to approximate the state-value function

$$v_{\phi}(s) \quad (9)$$

where  $\phi$  denotes the critic parameters. The critic shares the same architecture as the policy network (two hidden layers with 128 units and ReLU activations) but outputs a single scalar value representing the estimated return from state  $s$ .

**Update Mechanism:** Unlike the pure Monte Carlo method, the Actor–Critic approach uses bootstrapped returns. After each episode (or termination), the discounted return is computed recursively as

$$R_t = r_t + \gamma R_{t+1} \quad (10)$$

where  $R_{t+1}$  is replaced by the critic estimate of the next state if the episode has not terminated.

The advantage estimate is defined as

$$A_t = R_t - v_{\phi}(s_t) \quad (11)$$



The policy (actor) is updated by increasing the probability of actions with positive advantage, while the critic is trained to minimize the mean squared error between predicted values and computed returns. The combined loss is given by

$$\mathcal{L} = -\log \pi_{\theta}(a_t | s_t) A_t + (R_t - v_{\phi}(s_t))^2. \quad (12)$$

Both networks are optimized jointly using the Adam optimizer.

## V. EXPERIMENTAL SETUP

The experiments are conducted within the simulation environment described in Chapter III, which implements a two-dimensional N-body system. The velocity Verlet integrator was used because it is more sophisticated than the simpler Euler method, and for the short time frames considered in the experiments, the additional computational cost is negligible.

The initial scenario implemented in this project consisted of a simplified Hohmann transfer from a lower circular orbit to a higher circular orbit (See the Figure 1). This scenario provided a simple, physically interpretable benchmark for testing and evaluating the learning algorithms.

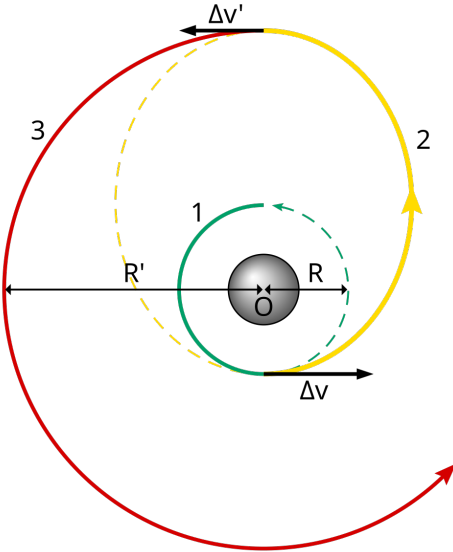


Fig. 1. Hohmann transfer orbit. Image by Leafnode, licensed under CC BY-SA 2.5, via Wikimedia Commons (source, accessed 19 February 2026).

### Hohmann Transfer Scenario

The simulation consists of a two-body system with a static Earth located at the origin and a spacecraft initialized in a circular orbit at an altitude of 2000 km. The initial velocity is chosen such that, in the absence of thrust, the spacecraft follows a stable circular orbit under Newtonian gravity.

For a circular orbit, the required orbital velocity is

$$v = \sqrt{\frac{\mu}{r}}, \quad (13)$$

where  $\mu = 3.986 \times 10^{14} \text{ m}^3/\text{s}^2$  is Earth's gravitational parameter and  $r$  is the distance from Earth's center. With

$$r_1 = R_E + 2000 \text{ km} \approx 8.371 \times 10^6 \text{ m}, \quad (14)$$

the resulting circular velocity is approximately

$$v_1 \approx 6.9 \times 10^3 \text{ m/s}. \quad (15)$$

The target orbit is defined at an altitude of 4000 km, resulting in

$$r_2 = R_E + 4000 \text{ km} \approx 1.0371 \times 10^7 \text{ m}, \quad (16)$$

with a circular velocity of approximately

$$v_2 \approx 6.2 \times 10^3 \text{ m/s}, \quad (17)$$

calculated using Eq. (13). The required velocity change is therefore  $\Delta v \approx 700 \text{ m/s}$ .

The spacecraft mass is set to  $m = 100,000 \text{ kg}$ . Control inputs are modeled as discrete thrust impulses applied at each integration step. The thrust magnitude corresponds to a constant force  $F = 10^6 \text{ N}$ . To ensure timestep invariance, the impulse applied at each step is computed as

$$J = F \cdot dt \quad (18)$$

The resulting velocity increment per action is

$$\Delta v_{\text{action}} = \frac{J}{m} = \frac{F \cdot dt}{m}. \quad (19)$$

For  $dt = 1 \text{ s}$  this yields  $\Delta v_{\text{action}} = 10 \text{ m/s}$ . This increment is small relative to  $\Delta v$  and therefore requires multiple coordinated thrust actions to complete the transfer. For simplicity, propellant mass depletion is neglected and the spacecraft mass is assumed constant.

At each time step, the agent selects a discrete thrust action and the spacecraft state is propagated using Newtonian orbital dynamics. An episode terminates when the target orbit is reached or when predefined bounds are violated (e.g., collision, escape, or maximum simulation length). The complete configuration parameters are listed in Appendix II.

### Observation Normalization

The learning agent observes the spacecraft state relative to the reference body (in this scenario Earth). Since orbital motion is governed by relative position and velocity, the observation space is constructed using relative state vectors  $\mathbf{r}_{\text{rel}}$  and  $\mathbf{v}_{\text{rel}}$  rather than absolute coordinates.

$$\mathbf{r}_{\text{rel}} = \mathbf{x}_{\text{ship}} - \mathbf{x}_{\text{ref}}, \quad (20)$$

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{ship}} - \mathbf{v}_{\text{ref}}. \quad (21)$$

Their magnitudes define the relative distance and speed:

$$d = \|\mathbf{r}_{\text{rel}}\|, \quad (22)$$

$$v = \|\mathbf{v}_{\text{rel}}\|. \quad (23)$$

To decouple directional information from magnitude and improve learning stability, the relative vectors are normalized:

$$\hat{\mathbf{r}}_{\text{rel}} = \frac{\mathbf{r}_{\text{rel}}}{d + \epsilon}, \quad (24)$$

$$\hat{\mathbf{v}}_{\text{rel}} = \frac{\mathbf{v}_{\text{rel}}}{v + \epsilon}. \quad (25)$$

where  $\epsilon$  is a small constant introduced to avoid division by zero.

In addition, the relative distance and speed are scaled with respect to the desired orbital parameters  $d_{\text{target}}$  and  $v_{\text{target}}$ :

$$\tilde{d} = \frac{d - d_{\text{target}}}{d_{\text{target}} + \epsilon}, \quad (26)$$

$$\tilde{v} = \frac{v - v_{\text{target}}}{v_{\text{target}} + \epsilon}. \quad (27)$$

A radial alignment term is further computed as the dot product between the normalized velocity and position vectors:

$$\alpha = \hat{\mathbf{v}}_{\text{rel}} \cdot \hat{\mathbf{r}}_{\text{rel}}. \quad (28)$$

This quantity represents the cosine of the angle between velocity and radial direction. Positive values indicate outward motion, negative values inward motion, and values close to zero correspond to tangential movement. For a stable circular orbit, which is the goal of the Hohmann transfer, the velocity vector should be perpendicular to the radial direction. Including this feature therefore directly supports learning orbit stabilization behavior.

The final observation vector consists of  $\hat{\mathbf{r}}_{\text{rel}}$ ,  $\hat{\mathbf{v}}_{\text{rel}}$ ,  $\tilde{d}$ ,  $\tilde{v}$ , and  $\alpha$ . This design separates directional information from magnitude information, in theory, allowing the neural network to learn both geometric orientation and orbital energy characteristics without numerical instability due to large physical scales.

### Reward Shaping

Similarly, reward shaping was designed specifically to reflect the goals of the Hohmann transfer scenario:

- Reaching the target orbital radius
- Matching the target circular orbital velocity
- Achieving tangential motion at the target orbit
- Minimizing total thrust usage
- Avoiding collisions or escape trajectories

The reward function provides positive feedback for reducing the distance to the target orbit and for aligning the spacecraft velocity tangentially to the desired circular motion. Additional penalties are applied for excessive thrust usage, collisions with the central body, or exceeding predefined system bounds such as escape distance or velocity.

To encourage timely convergence and discourage unnecessarily long maneuvers, a small constant penalty is applied at every simulation step. This introduces a mild time pressure that promotes efficient orbital transfer while avoiding excessively prolonged trajectories.

The shaping reward at each time step is defined as

$$r_{\text{shape}} = w_{\alpha}(1-|\alpha|) + w_d(1-|\tilde{d}|) + w_v(1-|\tilde{v}|) - w_{\text{time}} - w_{\text{thrust}}\mathbb{I}_{\text{thrust}}, \quad (29)$$

where  $\alpha$  is the radial alignment,  $\tilde{d}$  and  $\tilde{v}$  are normalized deviations from target,  $\mathbb{I}_{\text{thrust}}$  is 1 if thrust is applied and 0 otherwise. The specific reward weights  $w_i$  were iteratively adjusted across simulation runs to ensure stable training behavior and balanced learning dynamics.

Terminal conditions are calculated after the shaping reward and are defined as

$$r_{\text{terminal}} = \begin{cases} R_{\text{success}}, & \text{if orbital tolerances are satisfied} \\ -R_{\text{crash}}, & \text{if collision occurs} \\ -R_{\text{escape}}, & \text{if escape conditions are met.} \end{cases} \quad (30)$$

The total reward at time step  $t$  is

$$r_t = r_{\text{terminal}} + \frac{1}{N_{\text{max}}} \cdot r_{\text{shape}}. \quad (31)$$

where  $N_{\text{max}}$  is maximum episode length. The normalization factor

$$\frac{1}{N_{\text{max}}} \quad (32)$$

ensures that cumulative episode rewards remain bounded and approximately invariant with respect to episode duration, while terminal rewards maintain dominance to clearly signal success or failure. An overview of all shaping weights and terminal reward parameters is provided in Table III.

This configuration establishes a controlled experimental environment in which both Monte Carlo policy gradient and Advantage Actor-Critic algorithms can be evaluated. The normalization of observations and the shaping of rewards provide the agent with meaningful and interpretable signals.

## VI. RESULTS

This section presents the experimental results obtained for the Monte Carlo policy gradient and Advantage Actor-Critic algorithms. Each algorithm was trained for 10 independent runs to account for stochasticity in policy initialization and environment sampling. The evaluation focuses on training performance, success rate, convergence behavior, and qualitative trajectory analysis.

### Training Performance

The primary metric for evaluating learning progress is the episode return during training. Figures 2 and 3 show the mean episode return over 10 training runs for each algorithm, together with the corresponding standard deviation.

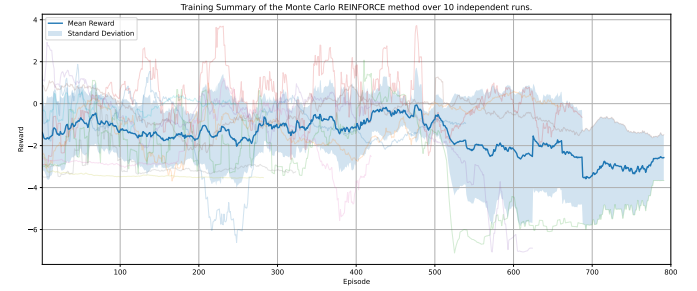


Fig. 2. Mean episode return and standard deviation over 10 training runs with the Monte Carlo REINFORCE method.

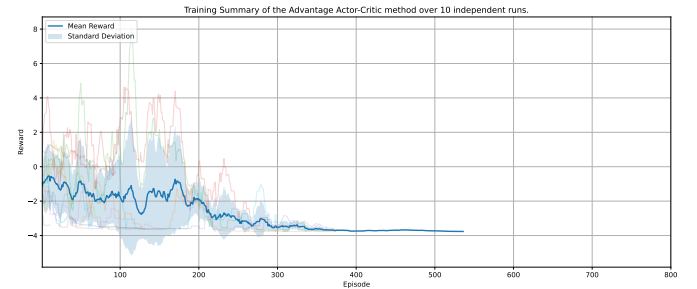


Fig. 3. Mean episode return and standard deviation over 10 training runs with the Advantage Actor-Critic method.

The shaded variance illustrates the stability of training across different random seeds.

### Success Rate

In addition to reward progression, the mean success rate across the 10 training runs was computed. A successful episode is defined as satisfying the terminal orbital tolerances without triggering collision or escape conditions.

Table I summarizes the average success rate and its standard deviation for both algorithms.

TABLE I

MEAN SUCCESS RATE OVER 10 TRAINING RUNS FOR EACH ALGORITHM.

| Algorithm              | Mean Success Rate | Std    |
|------------------------|-------------------|--------|
| Monte Carlo REINFORCE  | 0.96 %            | 1.41 % |
| Advantage Actor-Critic | 2.06 %            | 3.39 % |

### Convergence Behavior

Convergence is evaluated based on the smoothed episode return using an exponential moving average (EMA). For a sequence of episode returns  $R_k$ , the smoothed value is computed as

$$\text{EMA}_k = \alpha R_k + (1 - \alpha) \text{EMA}_{k-1}, \quad (33)$$

where  $\alpha = 0.1$  and the initialization is  $\text{EMA}_0 = R_0$ .

Convergence is defined as the earliest episode index at which the absolute change of the EMA falls below a predefined tolerance

$$|\text{EMA}_k - \text{EMA}_{k-1}| < 10^{-4} \quad (34)$$

for a sustained number of subsequent episodes.

The number of episodes and the corresponding wall-clock training time required to satisfy this condition are reported as convergence metrics. Table II summarizes the mean convergence episode and mean training time across the 10 independent training runs for each algorithm.

TABLE II

CONVERGENCE SPEED COMPARISON USING EMA-BASED CRITERION.

| Algorithm              | Mean Conv. Episode | Mean Conv. Time |
|------------------------|--------------------|-----------------|
| Monte Carlo REINFORCE  | 481                | 2025.3 seconds  |
| Advantage Actor-Critic | 261                | 795.7 seconds   |

### Behavioral Analysis

To evaluate the qualitative behavior of the learned policies, representative trajectories from successful episodes were selected for visualization. Figures 4 and 5 show the spacecraft trajectories for both algorithms compared to the target orbit.

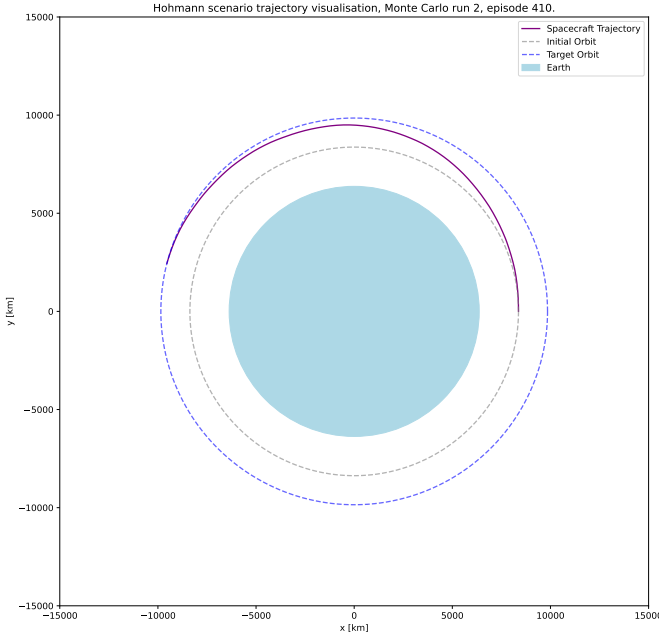


Fig. 4. Representative orbital trajectory generated by the Monte Carlo REINFORCE learning.

Additional plots of the discrete thrust commands over time (Figures 6 and 7) are provided to illustrate the learned control policy.

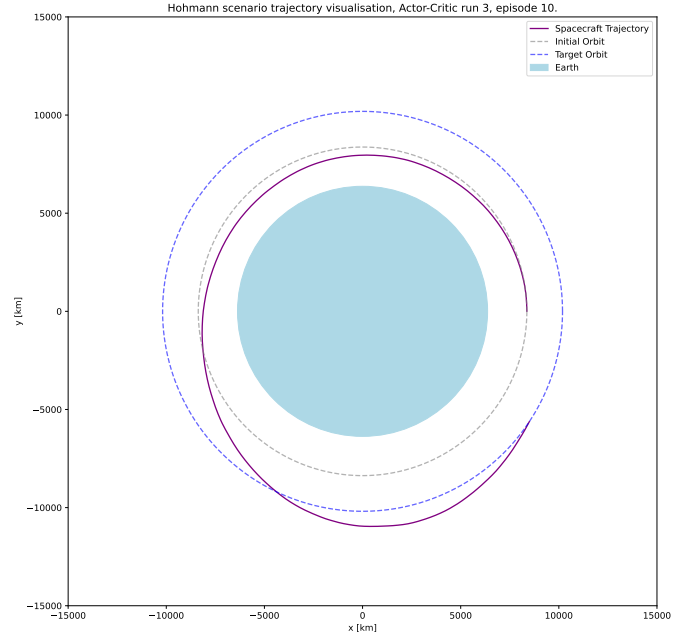


Fig. 5. Representative orbital trajectory generated by the Advantage Actor-Critic learning.

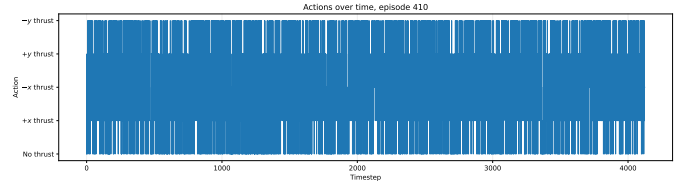


Fig. 6. Discrete thrust command sequence during the successful episode shown in Figure 4.

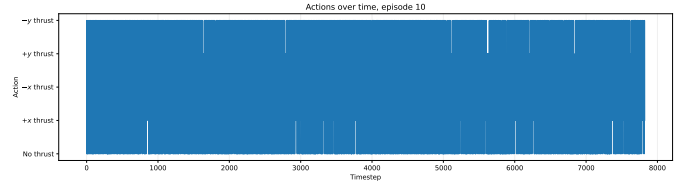


Fig. 7. Discrete thrust command sequence during the successful episode shown in Figure 5.

### Statistical Robustness

Since training is stochastic, performance variability across runs is important. Figure 8 shows the distribution of final episode returns across the 10 runs for each algorithm.

The observed variance provides insight into the robustness and reproducibility of each learning method.

## VII. DISCUSSION

This section interprets the experimental results and analyzes the observed limitations of the implemented reinforcement learning approaches.

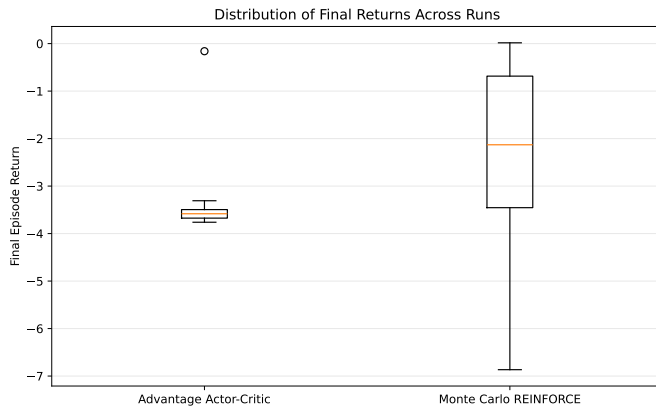


Fig. 8. Distribution of final episode returns across 10 independent training runs.

### Learning Behavior and Stability

Across all training runs, neither the Monte Carlo policy gradient method nor the Advantage Actor-Critic approach consistently achieved stable positive rewards corresponding to a physically meaningful Hohmann-like transfer. Instead, learned policies frequently converged to behaviors that triggered terminal failure conditions, such as collisions or escape from the system.

Learning behavior exhibited high variance across independent training runs, even under identical initial conditions and parameters. This indicates that the training process is sensitive to stochastic initialization and exploration noise. Furthermore, convergence to stable behavior was not consistently observed, suggesting that the current reward structure and state representation do not provide sufficiently strong guidance toward the desired orbital objective.

An additional observation is that the learned policies tend to produce rapidly changing thrust commands at every simulation step. Physically, optimal orbital transfers typically consist of impulsive maneuvers or structured thrust phases rather than continuously switching thrust directions. The absence of such structured impulse-like behavior indicates that the policy has not discovered physically meaningful control strategies.

### Algorithm Comparison

The Actor-Critic method demonstrated improved convergence speed and generally higher success rates compared to the Monte Carlo REINFORCE approach. This improvement is expected due to reduced gradient variance from bootstrapped value estimation.

However, despite improved computational efficiency and more stable updates, the Actor-Critic method still failed to reliably achieve stable orbit insertion. This suggests that algorithmic improvements alone are insufficient when the underlying reward formulation, observation design, or action parameterization limits effective policy learning.

### Potential Causes of Performance Limitations

Several factors may have contributed to the observed training behavior:

- **Reward shaping limitations:** The reward function may not provide sufficiently informative intermediate guidance toward the optimal transfer trajectory. Sparse or poorly balanced reward components can lead to local optima or trivial policies.
- **Observation representation:** Although the observation space includes normalized relative position and velocity information,

it may not contain sufficient global context for long-horizon planning. Additional features such as orbital energy, angular momentum, or historical state information might improve policy performance.

- **Action space design:** The thrust action space is defined in fixed global directions rather than relative to the spacecraft's instantaneous frame. This may hinder efficient maneuver learning, since optimal thrust for orbital transfers is typically aligned with velocity or radial directions.
- **Hyperparameter Sensitivity:** The policy and value networks consist of two hidden layers with 128 ReLU units. While this architecture provides moderate representational capacity, no systematic tuning was performed. The networks may therefore be either insufficiently expressive for the nonlinear dynamics or overly flexible, leading to unstable gradient updates. Several critical hyperparameters were fixed without extensive exploration, including the learning rate, discount factor  $\gamma$ , network size, and activation functions. In particular,  $\gamma$  strongly influences long-term planning. Since orbital transfers require delayed reward optimization, suboptimal discounting may bias the agent toward short-term behaviors. Exploration is induced solely through sampling from a categorical distribution over discrete thrust directions. In highly sensitive dynamical systems, such stochastic exploration may be insufficient to reliably discover structured impulsive maneuver sequences.

### Simulation and Modeling Considerations

Beyond algorithmic limitations, several aspects of the physical simulation have to be considered for more complex scenarios:

- **Uniform timestep integration:** The simulation currently employs a single fixed timestep for all bodies. While computationally convenient, this approach reduces numerical accuracy for fast-moving or closely interacting objects. More accurate modeling could require adaptive timesteps based on orbital period or instantaneous velocity.
- **Initial configuration assumptions:** The initial orbital parameters assume textbook circular orbits with simplified alignment conditions at  $t = 0$ . However, these parameters are often derived in different inertial reference frames than the simulation environment. For realistic scenarios (e.g., Earth-centered orbits), initial conditions must be corrected to the appropriate frame of reference to ensure physical consistency.
- **Integration method limitations:** Although the velocity Verlet integrator provides improved stability compared to simpler explicit methods, more complex or long-range orbital scenarios may require higher-order integration schemes such as Runge-Kutta methods (e.g., RK4) to improve accuracy for long-duration simulations.

### Overall Interpretation

The results indicate that reinforcement learning can produce partially structured control behaviors within the defined environment, but fails to consistently generate stable and physically meaningful orbital transfer trajectories under the current formulation. While both REINFORCE and Actor-Critic demonstrate measurable learning dynamics, convergence remains unstable and final performance does not reach sustained positive reward regimes.

The project evaluates baseline policy-gradient approaches under a fixed architectural and hyperparameter configuration in order to analyze their behavior in a simplified orbital transfer scenario. Although no extensive hyperparameter optimization was conducted, the



experiments provide a controlled comparison between Monte Carlo and Actor–Critic methods within the proposed environment.

The findings therefore should not be interpreted as evidence that reinforcement learning is unsuitable for orbital control. Rather, they suggest that the current problem formulation, including reward design, state representation, action parameterization, and model configuration, may not yet sufficiently align with the complexity of long-horizon dynamical planning. More advanced policy-gradient methods, such as Proximal Policy Optimization (PPO), which introduce clipped surrogate objectives and improved training stability, remain to be implemented and systematically evaluated in this setting.

## VIII. CONCLUSION AND FUTURE WORK

This work explored the feasibility of applying reinforcement learning to a simplified orbital transfer task. A custom simulation environment was designed to model spacecraft motion under gravitational dynamics, and two policy-gradient methods, REINFORCE and Actor–Critic, were implemented and systematically evaluated.

Although neither approach achieved stable and fully successful orbital transfers, both methods demonstrated the ability to learn partial control behaviors. The Actor–Critic method showed improved convergence speed and higher average success rates compared to the Monte Carlo baseline, indicating the advantage of value-function guidance in long-horizon dynamical tasks.

Importantly, this project established a modular and extensible experimental framework that integrates physics simulation, training pipelines, logging mechanisms, and performance evaluation metrics. This framework provides a foundation for continued research and algorithmic refinement.

Future improvements should focus on several key directions. First, the reward structure should be refined to provide clearer intermediate guidance toward physically meaningful transfer trajectories. Second, the observation representation may be enhanced by incorporating higher-level orbital features, such as energy or angular momentum, to better reflect underlying mechanics. Third, the action parameterization should be reconsidered to align thrust directions with physically relevant reference frames, potentially enabling more efficient maneuver learning.

In addition, systematic hyperparameter tuning and architectural exploration are necessary to better match model capacity with the complexity of the task. More advanced policy optimization methods, particularly Proximal Policy Optimization (PPO), should be implemented and evaluated to improve training stability and sample efficiency.

Overall, while stable optimal transfers were not achieved under the current configuration, the results demonstrate both the challenges and potential of reinforcement learning for orbital control problems. The developed framework enables further experimentation toward more robust and physically consistent autonomous spaceflight control strategies.

## CODE AVAILABILITY

The complete source code for the simulation environment, reinforcement learning implementations, experiment evaluation and visualization scripts is publicly available at: <https://github.com/DanyBoi22/AstroRL.git>

## REFERENCES

- [1] B. Weber, “Hohmann transfer,” [Online]. Available: <https://orbital-mechanics.space/orbital-maneuvers/hohmann-transfer.html>
- [2] D. Izzo, “Revisiting lambert’s problem,” *Celestial Mechanics and Dynamical Astronomy*, vol. 121, no. 1, pp. 1–15, 2015.
- [3] Y. Li, M. Ono *et al.*, “Smooth and continuous interplanetary trajectory design of spacecraft using iterative patched-conic method,” *Acta Astronautica*, vol. 185, pp. 58–70, 2021.
- [4] F. Peralta and S. Flanagan, “Cassini interplanetary trajectory design,” *Control Engineering Practice*, vol. 3, no. 11, pp. 1603–1610, 1995. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S096706619500171P>
- [5] D. Woffinden, R. Eckman, and S. Robinson, “Optimized trajectory correction burn placement for the NASA Artemis II mission,” in *45th Annual AAS Guidance, Navigation and Control Conference*, Breckenridge, CO, USA, 2023, aAS 23-062.
- [6] A. Zavoli and L. Federici, “Reinforcement learning for robust trajectory design of interplanetary missions,” *Journal of Guidance, Control, and Dynamics*, vol. 44, no. 7, pp. 1312–1326, 2021.
- [7] A. Jain, V. Rodriguez-Fernandez, and R. Linares, “Multi-phase spacecraft trajectory optimization via transformer-based reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.11402>
- [8] C. Schäfer, “N-body simulations with REBOUND,” Universität Tübingen, Institut für Astronomie und Astrophysik, Tech. Rep., sep 2024, lecture notes and tutorial. [Online]. Available: [https://www.tat.physik.uni-tuebingen.de/~schaefer/teach/f/rebound\\_applications.pdf](https://www.tat.physik.uni-tuebingen.de/~schaefer/teach/f/rebound_applications.pdf)
- [9] J. Barrow-Green, “The three-body problem,” *The Princeton Companion to Mathematics*, pp. 726–728, 2008.
- [10] Z. E. Musielak and B. Quarles, “The three-body problem,” *Reports on Progress in Physics*, vol. 77, no. 6, p. 065901, jun 2014. [Online]. Available: <https://doi.org/10.1088/0034-4885/77/6/065901>
- [11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>

## APPENDIX I

### ENVIRONMENT SIMULATION VISUALIZATIONS AND STABILITY ANALYSIS

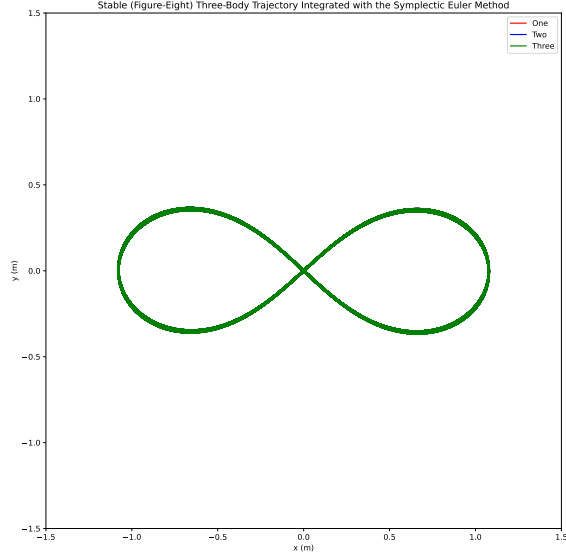


Fig. 9. Trajectory of a three-body system on a stable "figure-eight" orbit over 10,000,000 simulation steps with a time step of  $\Delta t = 0.001$  s, integrated using the Symplectic Euler method.

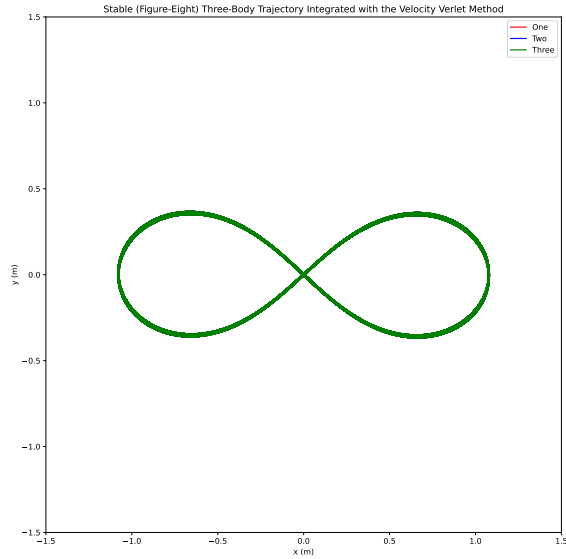


Fig. 10. Trajectory of a three-body system on a stable "figure-eight" orbit over 10,000,000 simulation steps with a time step of  $\Delta t = 0.001$  s, integrated using the Velocity Verlet method.

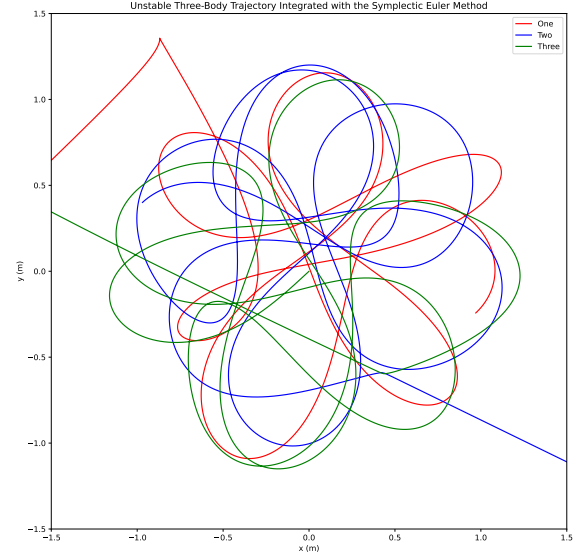


Fig. 11. Trajectory of a three-body system on an unstable orbit over 10,000 simulation steps with a time step of  $\Delta t = 0.001$  s, integrated using the Symplectic Euler method.

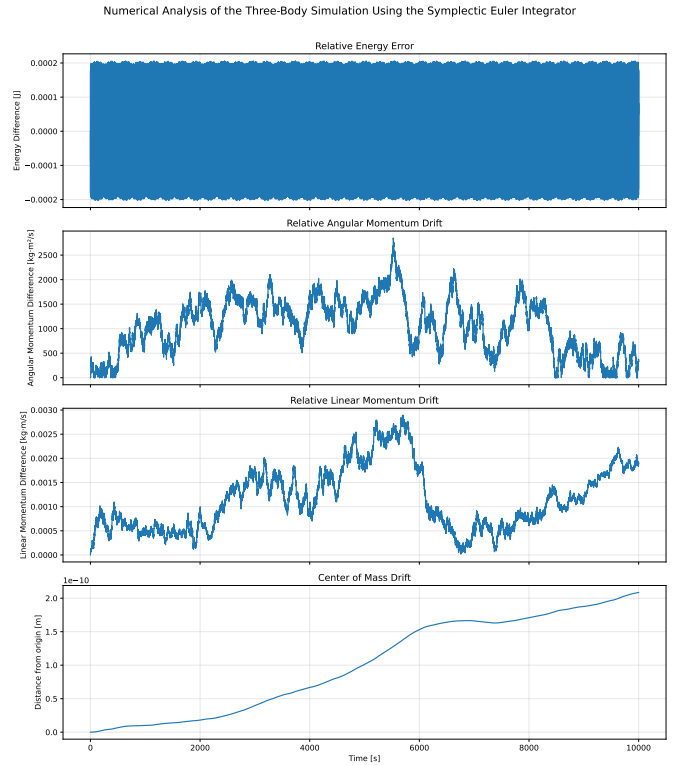


Fig. 12. Assessment of energy and momentum conservation of a three-body system on a stable "figure-eight" orbit over 10,000,000 simulation steps with a time step of  $\Delta t = 0.001$  s, integrated using the Symplectic Euler method.

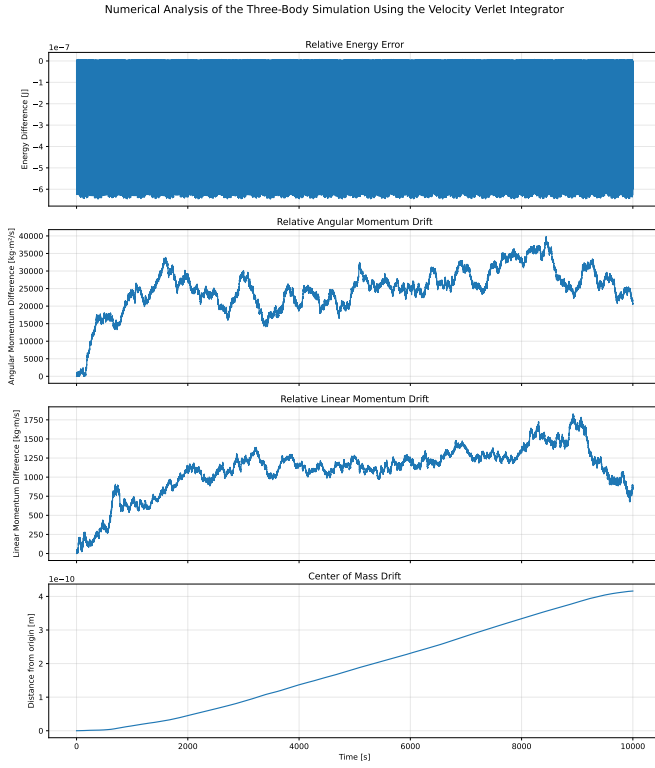


Fig. 13. Assessment of energy and momentum conservation of a three-body system on a stable "figure-eight" orbit over 10,000,000 simulation steps with a time step of  $\Delta t = 0.001$  s, integrated using the Velocity Verlet method

## APPENDIX II ENVIRONMENT AND SCENARIO CONFIGURATION

LISTING 1. Example of environment configuration parameters used for the Hohmann transfer scenario.

```
earth_radius = 6.371e6
bodies = [
    np.array([          # Masses in kg
        5.972e24,      # Earth
        1.0e5,         # Ship
    ]),
    np.array([          # Positions (x, y) in meters
        [0.0, 0.0],
        [earth_radius + 2e6, 0.0],
    ]),
    np.array([          # Velocities (vx, vy) in m/s
        [0.0, 0.0],
        [0.0, 6.9e3],
    ]),
    np.array([          # Names of bodies
        "Earth",
        "Ship"
    ]),
    np.array([          # Colors for visuals
        "blue",         # Earth
        "lightblue",    # Ship
    ]),
]
forcemodel = GravityForce
integrator = VelocityVerlet
ship = SimpleImpulseShip
```

LISTING 2. Example of training configuration parameters used for the Hohmann transfer scenario.

```
dt = 1          # seconds, stable sim
simulation_time = 10500 * 1 * 1
# seconds, full rotation on target orbit
max_steps = int(simulation_time / dt)

ship_index = len(bodies[0])-1
mass = bodies[0][ship_index]
reward_coef = 1.0 / max_steps
thrust = 1e6 * dt

training_config = TrainingConfig(
    bodies=bodies,
    dt=dt,
    max_steps=max_steps,
    n_episodes=800,
    log_every=10,
    log_states=True,
    log_n_entries=400,
    ship_index=ship_index,
    mass=mass,
    thrust=thrust,
    actions=simple_action_space,
    safety_radius=earth_radius + 1e5, # atmo entry
    escape_dist=1e8,
    escape_vel=1e5,
    target_dist=earth_radius + 4e6,
    target_vel=6.2e3,
    reference_point_index=0,
    reward_coef=reward_coef
)
agent = MonteCarloAgent
```

TABLE III  
REWARD PARAMETERS USED IN THE REINFORCEMENT LEARNING FORMULATION.

| Category                | Parameter            | Value |
|-------------------------|----------------------|-------|
| <i>Shaping Weights</i>  |                      |       |
| Alignment weight        | $w_\alpha$           | 1     |
| Distance weight         | $w_d$                | 2     |
| Velocity weight         | $w_v$                | 0.3   |
| Time penalty            | $w_{\text{time}}$    | 0.2   |
| Thrust penalty          | $w_{\text{thrust}}$  | 0.2   |
| <i>Terminal Rewards</i> |                      |       |
| Success reward          | $R_{\text{success}}$ | 10    |
| Crash penalty           | $R_{\text{crash}}$   | 4     |
| Escape penalty          | $R_{\text{escape}}$  | 4     |